PROJECT GUIDE · v1.0

ল্যাব ইনফরমেশন সিস্টেম (LIS)

Analyzer → Middleware → Oracle Database → Oracle APEX

আপনার ল্যাবের মেশিন (Sysmex, Mindray, Roche, Abbott) থেকে রিপোর্ট সরাসরি Oracle ডেটাবেসে আনার সম্পূর্ণ সমাধান — কোড, আর্কিটেকচার, সেটআপ, টেস্ট আর troubleshooting সহ।

স্তর	বিস্তারিত
Frontend	Oracle APEX — রিপোর্ট, ড্যাশবোর্ড, critical alert
Database	Oracle 19c / 21c / 23ai — thin mode, কোনো Oracle Client দরকার নেই
নিয়ন্ত্রণ	সব কনফিগ APEX পেজ থেকে — Python সার্ভারে কেউ ঢুকবে না
Middleware	Python 3.10+ — ASTM ও HL7 parser, TCP ও Serial listener
Analyzer	Sysmex XN (ASTM/TCP), Mindray BS-240 (HL7/TCP), Roche Cobas (ASTM/Serial)
ডেমো	ব্রাউজারে চলা mock analyzer — real machine ছাড়াই পুরো flow টেস্ট করা যায়

এই ডকুমেন্টে যা পাবেন

1) কেন middleware লাগে 2) পুরো আর্কিটেকচার 3) ফাইল ও কোড 4) ডেটাবেস স্কিমা 5) APEX রিপোর্ট ও APEX-চালিত কন্ট্রোল 6) সেটআপ ধাপে ধাপে 7) টেস্ট ও মক অ্যানালাইজার 8) ডিপ্লয়মেন্ট 9) সমস্যা ও সমাধান 10) চেকলিস্ট

সূচিপত্র

#	বিভাগ	কী আছে
১	কেন মিডলওয়্যার দরকার	Mirth নাকি নিজের কোড — সিদ্ধান্তের ভিত্তি
২	আর্কিটেকচার	পাঁচ স্তর, প্রতিটির দায়িত্ব
৩	কোড	ASTM/HL7 ফরম্যাট, pipeline, ACK, ডুপ্লিকেট
৪	ডেটাবেস স্কিমা	টেবিল, ইনডেক্স, APEX ভিউ
৫	Oracle APEX	রিপোর্ট, ফর্ম, critical alert + APEX থেকে সব নিয়ন্ত্রণ
৬	সেটআপ ও ল্যাব ডে	স্কিমা → config → preflight → মেশিন সেটিং → ল্যাবের ক্রম
৭	টেস্ট	৫০টি ইউনিট টেস্ট, মক অ্যানালাইজার, ব্রাউজার ডেমো
৮	ডিপ্লয়মেন্ট	Windows সার্ভিস, Linux systemd
৯	সমস্যা ও সমাধান	লক্ষণ → কারণ → সমাধান
১০	চেকলিস্ট	প্রোডাকশনে যাওয়ার আগে

গুরুত্বপূর্ণ: এই সফটওয়্যার রেজাল্ট শুধু পরিবহন ও সংরক্ষণ করে — কোনো ভ্যালু নিজে হিসাব করে না। প্রতিটি analyzer-এর manual আর ল্যাবের SOP অনুযায়ী validation করুন, তারপর প্রোডাকশনে দিন।

বিভাগ ১

কেন মিডলওয়্যার দরকার

ল্যাব অ্যানালাইজার Oracle ডেটাবেস চেনে না। মেশিন একটা সিরিয়াল কেবল বা LAN কেবল দিয়ে শুধু **টেক্সট** পাঠায় — ASTM (E1394) অথবা HL7 v2। Oracle চায় **SQL INSERT**। দুই ভাষার মাঝখানে যে অনুবাদক বসে, সেটাই মিডলওয়্যার।

একই রেজাল্ট, দুই রকম ফরম্যাট

```
ASTM (মেশিন যা পাঠায়)                    SQL (ডেটাবেস যা বোঝে)
---------------------------------------     --------------------------------
H|\^&|||Sysmex^XN-1000|||||||...             INSERT INTO lab_results (
P|1||2024001234||Karim, Ahmed||19850615|M        sample_id, test_code,
O|1|SAMPLE-0001||^^^CBC|R                       result_value, result_unit
R|1|^^^WBC^6690-2|7.5|10*3/uL|4.0-11.0|N|F    ) VALUES (
L|1|N                                           'SAMPLE-0001', 'WBC', 7.5, '10*3/uL'
                                            );
```

১.১ তিনটি পথ — কোনটা আপনার জন্য

বিকল্প	সুবিধা	অসুবিধা	কখন ভালো
Mirth Connect	ফ্রি কোর, HL7+MLLP নেটিভ, GUI	ASTM E1394/E1381 আলাদা কমার্শিয়াল এক্সটেনশন; Java দরকার	HL7-ভাষী analyzer, একাধিক মেশিন
নিজের কোড (এই প্রজেক্ট)	পূর্ণ নিয়ন্ত্রণ, কোনো লাইসেন্স নেই, Oracle-native	নিজে মেইনটেইন করতে হবে	১-৩টি analyzer, কোড শেখার ইচ্ছা
Vendor middleware	ভেন্ডর সাপোর্ট	ব্যয়বহুল, একই ব্র্যান্ডে সীমাবদ্ধ	এন্টারপ্রাইজ, বাজেট আছে

সিদ্ধান্ত
আপনি নিজের কোড চেয়েছেন — তাই এই প্রজেক্টে **সম্পূর্ণ Python middleware** বানানো হয়েছে। Mirth-এর মতোই কাজ করে, কিন্তু কোনো তৃতীয় পক্ষের উপর নির্ভর নেই।

বিভাগ ২

আর্কিটেকচার

পাঁচটি স্তর। প্রতিটি স্তরের কাজ আলাদা ফাইলে, তাই একটা বদলালে বাকিগুলো ভাঙে না।

ডেটা প্রবাহ (এক দিক) + ACK (ফেরত পথ)

```
MACHINES                          PROTOCOL / PORT
  Sysmex XN-1000                  ASTM over TCP  :6661
  Mindray BS-240                  HL7 over TCP   :6662
  Roche Cobas c311                ASTM / RS-232  COM3
      |  raw text payload (ASTM or HL7)
      v
LIS MIDDLEWARE (this project)
  1 listener   accept, strip MLLP/ASTM framing, one task per analyzer
  2 parser     text -> structured result dicts (ASTM / HL7)
  3 validator  required fields, flags, test-code mapping
  4 db writer  duplicate check, INSERT, raw archive
  5 ack        reply so the machine keeps sending
      |  SQL (python-oracledb, thin mode)
      v
ORACLE DATABASE   lis_raw_messages  +  lab_results
      |  SELECT / views
      v
ORACLE APEX       reports, dashboard, critical alerts
```

২.১ প্রতিটি স্তর কী করে

স্তর	ফাইল	দায়িত্ব
Listener	listeners/tcp_listener.py listeners/serial_listener.py	পোর্ট খোলা রাখা, একাধিক মেশিন একসাথে সামলানো, MLLP/ASTM ফ্রেমিং ভাঙা, ACK পাঠানো
Parser	parsers/astm_parser.py parsers/hl7_parser.py	টেক্সট → স্ট্রাকচার্ড dict: sample, patient, test code, value, unit, range, flag
Validator	core/validator.py	বাধ্যতামূলক ফিল্ড, flag ঠিক করা (H/L/HH/LL), critical শনাক্ত, test code mapping
Writer	database/db_writer.py	ডুপ্লিকেট চেক করে INSERT, raw message সংরক্ষণ, audit log
Engine	core/engine.py	উপরের সব ধাপ সঠিক ক্রমে সাজানো — একমাত্র জায়গা যেখানে order বোঝা যায়

> **নকশার মূল সিদ্ধান্ত**
> **raw message আগে সংরক্ষণ, তারপর parse।** কোনো কারণে parser ব্যর্থ হলে ডেটা হারায় না — পরে tools/replay.py দিয়ে একই মেসেজ আবার প্রসেস করা যায়।

বিভাগ ৩

কোড — ফাইল ধরে ধরে

৩.১ প্রজেক্ট স্ট্রাকচার

```
lis-middleware/
├── service.py              # production: APEX-driven + hot reload
├── main.py                 # dev: start every listener once
├── config/config.yaml      # DB host/user, log level, fallback list
├── core/
│   ├── engine.py           # raw → parse → validate → insert → ACK
│   ├── parser_factory.py   # protocol sniff + parser selection
│   ├── validator.py        # rules + normalisation
│   └── ack_handler.py      # ASTM 0x06 ACK / MLLP ACK builder
├── parsers/{astm_parser,hl7_parser}.py
├── listeners/{tcp_listener,serial_listener}.py
├── database/{db_connection,db_writer,control}.py
│   + schema.sql            # results tables
│   + apex_install.sql      # APEX control plane (lis_config ...)
├── demo/app.py             # browser demo (same engine)
├── ops/                    # systemd / NSSM service installers
├── tools/{device_sim,preflight,sniff_com,replay_capture}.py
├── samples/messages.yaml   # real-shaped ASTM / HL7 payloads
└── tests/test_pipeline.py  # 50 tests, no Oracle needed
```

৩.২ ASTM মানে কী — এক নজরে

ASTM E1394 মেসেজের প্রতিটি লাইন একেকটা **record**। প্রথম অক্ষরই বলে দেয় এটা কী:

Record	অর্থ	আমাদের দরকার?
H	Header — মেশিনের নাম, সময়	হ্যাঁ (instrument, timestamp)
P	Patient — রোগীর আইডি, নাম, বয়স	হ্যাঁ
O	Order — স্যাম্পল আইডি, কোন টেস্ট	হ্যাঁ
R	Result — টেস্ট কোড, ভ্যালু, ইউনিট, রেঞ্জ, flag	হ্যাঁ — মূল ডেটা
C / L	Comment / Terminator	না

একটা R লাইন ভেঙে দেখলে:

ASTM R record-এর ফিল্ড ম্যাপিং

```
R|1|^^^WBC^6690-2|7.5|10*3/uL|4.0-11.0|N||F
| | |            |   |        |        | |  └─ F = final
| | |            |   |        |        | └──── abnormal flag (N/H/L/HH/LL)
| | |            |   |        |        └─────── reference range
| | |            |   |        └──────────────── unit
| | |            |   └───────────────────────── value
| | └────────────┴──────────────────────────── ^^^TEST^LOINC  (কোড ৪র্থ অংশে)
| └──────────────────────────────────────────── record serial
└────────────────────────────────────────────── record type
```

৩.৩ HL7 মানে কী

HL7 v2 হলো segment-ভিত্তিকি: প্রতিটি লাইন `SEG|field1|field2...`। রেজাল্ট থাকে **OBX**-এ, রোগীর তথ্য **PID**-এ, স্যাম্পল **OBR**-এ। MSH দিয়ে শুরু, তাই MSH-1 ফিল্ডটাই separator — এখানেই বেশিরভাগ মানুষ এক ঘর করে ভুল করে।

```
MSH|^~\&|MINDRAY|BS240|LISMW|HOSPITAL|20240115110000||ORU^R01|MSG00042|P|2.5.1
PID|1||2024009012||Akter^Fatema||19780311|F          ← রোগী
OBR|1||SAMPLE-0003|CHEM7                             ← স্যাম্পল
OBX|1|NM|GLU^Glucose^2345-7||5.4|mmol/L|3.9-5.5|N|||F
               |  |     |       |   |      |       └─ flag
               |  |     |       |   |      └─────── range
               |  |     |       |   └────────────── unit
               |  |     |       └────────────────── value
               |  └─────┴──────────────────────────── CODE^TEXT^LOINC
               └─────────────────────────────────────── value type (NM/ST/TX)
```

৩.৪ Pipeline — core/engine.py

পুরো সিস্টেমের হৃদয়। ছয়টি ধাপ, প্রতিটির আগে-পরে লগ থাকে:

৬ ধাপের পাইপলাইন — ব্যর্থ হলেও raw ডেটা থাকে

```
def process(self, raw, analyzer, peer="n/a"):
    message_id = self.store.log_raw_message(...)   # 1. keep raw payload
    protocol   = sniff_protocol(raw)               # 2. ASTM or HL7?
    parsed     = parse_message(raw, ...)           # 3. text -> dicts
    validated  = self.validator.validate(parsed)   # 4. apply rules
    stored     = self.store.insert_results(...)    # 5. dedupe + INSERT
    ack        = self.ack.build(ok)                # 6. reply to analyzer
    return {...}
```

৩.৫ কেন ACK এত গুরুত্বপূর্ণ

বেশিরভাগ অ্যানালাইজার ACK না পেলে **পরের স্যাম্পল পাঠায় না** — মেশিন আটকে যায়। দুই রকম ACK আছে, analyzer-এর manual দেখে বেছে নিতে হয়:

Mode	কী পাঠাতে হয়	config
ASTM (E1381)	এক বাইট: 0x06 (ACK) বা 0x15 (NAK)	app.ack_mode: ASTM
HL7 MLLP	পূর্ণ MSA সেগমেন্ট সহ একটা মেসেজ, \x0b...\x1c\r	app.ack_mode: MLLP

৩.৬ ডুপ্লিকেট কেন হয়, কীভাবে থামে

মেশিন একই রেজাল্ট দুইবার পাঠায় — নেটওয়ার্ক টাইমআউট হলে, বা অপারেটর আবার স্যাম্পল চালালে। তিন স্তরে সুরক্ষা: **INSERT-এর আগে COUNT চেক**, **UNIQUE constraint** (sample_id, test_code, analyzer_name, result_value), আর **raw status = DUPLICATE**।

database/db_writer.py — is_duplicate()

```
SELECT COUNT(*) FROM lab_results
 WHERE sample_id = :sid AND test_code = :tc
   AND analyzer_name = :an AND result_value = :val;
-- when count > 0: log a warning, skip the INSERT
```

বিভাগ ৪

ডেটাবেস স্কিমা

তিনটি টেবিল + একটি status টেবিল। সবকিছু `database/schema.sql`-এ আছে, একবার চালালেই তৈরি হয়ে যায়।

৪.১ lis_raw_messages — কাঁচা মেসেজ

```
CREATE TABLE lis_raw_messages (
  message_id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  analyzer_name VARCHAR2(100) NOT NULL,
  protocol      VARCHAR2(10),          -- ASTM / HL7
  peer          VARCHAR2(100),         -- 192.168.1.50:51244
  raw_message   CLOB,
  received_at   TIMESTAMP DEFAULT SYSTIMESTAMP,
  status        VARCHAR2(20),          -- RECEIVED/PARSED/DUPLICATE/ERROR
  note          VARCHAR2(500)
);
```

এই টেবিল কখনো খালি করবেন না — parser-এ বাগ ঠিক করার পর পুরনো মেসেজ এখান থেকেই আবার প্রসেস হয়।

৪.২ lab_results — রিপোর্টের মূল টেবিল

```
CREATE TABLE lab_results (
  result_id       NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  sample_id       VARCHAR2(50)  NOT NULL,
  patient_id      VARCHAR2(50),
  patient_name    VARCHAR2(150),
  test_code       VARCHAR2(30)  NOT NULL,   -- WBC, HGB, GLU ...
  result_value    NUMBER,                   -- numeric হিসাবের জন্য
  result_text     VARCHAR2(100),            -- '<150', 'Positive' ইত্যাদি
  result_unit     VARCHAR2(30),
  ref_range       VARCHAR2(50),
  abnormal_flag   VARCHAR2(5),              -- N/H/L/HH/LL/C
  is_abnormal     CHAR(1),                  -- APEX-এ ফিল্টার সহজ করতে
  is_critical     CHAR(1),
  analyzer_name   VARCHAR2(100) NOT NULL,
  result_datetime TIMESTAMP,
  received_at     TIMESTAMP DEFAULT SYSTIMESTAMP,
  CONSTRAINT uq_result UNIQUE (sample_id, test_code, analyzer_name, result_value)
);
```

৪.৩ APEX-এর জন্য ভিউ

ভিউ	কী দেয়	APEX-এ কোথায়
v_sample_results	প্রতি স্যাম্পলের সারসংক্ষেপ + abnormal সংখ্যা	Result Entry পেজের রিপোর্ট
v_critical_results	শুধু critical রেজাল্ট, নতুন আগে	ড্যাশবোর্ড, auto-refresh
v_analyzer_throughput	ঘণ্টাভিত্তিক মেশিনভেদে রেজাল্ট সংখ্যা	Monitoring পেজ

বিভাগ ৫

Oracle APEX রিপোর্ট

Middleware ডেটা ঢোকানোর পর APEX-এ মাত্র তিনটি জিনিস বানালেই পুরো সিস্টেম ব্যবহারযোগ্য:

1. **Interactive Report** — source হিসেবে `SELECT * FROM v_sample_results` (বা lab_results)। samp_id-এ লিংক দিয়ে ডিটেইল পেজে যাবে।
2. **Form পেজ** — রোগী/স্যাম্পল রেজিস্ট্রেশন, বিক্রয় বা বারকোড স্ক্যান। এখানে যা সেভ হবে সেই sample_id-ই মেশিন থেকে ফিরে আসবে।
3. **Classic Report + Alert** — `v_critical_results` উপর ভিত্তি করে; রিফ্রেশ ৩০ সেকেন্ড, নতুন critical এলে লাল ব্যাজ।

```
-- one patient: every result, sorted by test code
SELECT test_code, result_value, result_unit, ref_range, abnormal_flag
  FROM lab_results
 WHERE sample_id = :P10_SAMPLE_ID
 ORDER BY test_code;

-- is data still arriving? (quick health check per analyzer)
SELECT analyzer_name, MAX(received_at) AS last_seen,
       COUNT(*) AS results_today
  FROM lab_results
 WHERE received_at >= TRUNC(SYSDATE)
 GROUP BY analyzer_name;
```

> **ছোট টিপস**
> APEX-এ রিপোর্ট পেজে **Pagination → 50** আর **Column → result_value → Number format** দিয়ে দিলে ল্যাবের লোকজন সহজে পড়তে পারবে। abnormal_flag-এ Highlight রঙ দিলে H/L চোখে পড়ে।

৫.৬ সব নিয়ন্ত্রণ APEX থেকে — ল্যাব কখনো সার্ভারে ঢুকবে না

উপরের রিপোর্টগুলো শুধু **দেখার** অংশ। কিন্তু আসল প্রশ্নটা হলো: অ্যানালাইজার যোগ করা, পোর্ট বদলানো, একটা মেশিন সাময়িক বন্ধ করা — এসব **কে** করবে? প্রচলিত উত্তর: সার্ভারে গিয়ে `config.yaml` এডিট, তারপর সার্ভিস রিস্টার্ট। এতে তিনটি সমস্যা: (ক) ল্যাবের লোক সার্ভারে ঢুকতে চায় না, ঢোকা উচিতও নয়; (খ) কে কখন কী বদলেছে তার হিসাব থাকে না; (গ) একটা অক্ষর ভুল হলে পুরো সার্ভিস বন্ধ, অথচ অ্যানালাইজার তখনো নমুনা পাঠাচ্ছে।

তাই এই ডিজাইনে **Oracle টেবিলই একমাত্র সত্য**। APEX পেজ শুধু ওই টেবিলের ফর্ম; Python সার্ভিস প্রতি ৫ সেকেন্ডে টেবিল পড়ে বদলটা নিজে থেকে প্রয়োগ করে, আর সফল/ব্যর্থতা টেবিলেই ফিরিয়ে লেখে।

```
APEX form (lab staff)
      |  INSERT / UPDATE
      v
  lis_config / lis_commands          <-- the only interface
      |  poll every 5 s
      v
  service.py  ->  apply diff  ->  restart ONLY the changed listener
      |
      +-->  lis_analyzer_status (ONLINE, last message)
      +-->  lis_config.last_applied_at / last_error  --> green tick in APEX
```

৫.৬.১ একবারের ইনস্টল

1. `sqlplus lis_user/pass@//host:1521/ORCL @database/schema.sql` — রেজাল্ট টেবিল ও ভিউ।
2. `sqlplus lis_user/pass@//host:1521/ORCL @database/apex_install.sql` — কন্ট্রোল টেবিল (`lis_config`, `lis_test_code_map`, `lis_commands`, `lis_analyzer_status`), APEX-এর ভিউ (`v_lis_analyzers`, `v_lis_settings`, `v_lis_health`, `v_lis_command_log`) এবং তিনটি ডেমো অ্যানালাইজার + পাঁচটি সেটিং সিড।
3. APEX-এ পেজ বানান (`docs/APEX_CONTROL.md` §২-এ প্রতিটি ফিল্ড, LOV আর নিয়ম লেখা আছে)।
4. সার্ভারে সার্ভিস ইনস্টল করুন — `ops/` ফোল্ডারের স্ক্রিপ্ট, নিচে দেখুন।

৫.৬.২ APEX পেজ ম্যাপ

#	পেজ	Region source	কাজ
১	অ্যানালাইজার	v_lis_analyzers	তালিকা: অনলাইন/অফলাইন, শেষ মেসেজ
২	অ্যানালাইজার ফর্ম	lis_config	পোর্ট, প্রোটোকল, সিরিয়াল সেটিং
৩	সেটিংস	v_lis_settings	ACK mode, log level, QC prefix
৪	টেস্ট কোড ম্যাপ	lis_test_code_map	মেশিনের কোড → রিপোর্ট কোড
৫	কন্ট্রোল বাটন	lis_commands	Reload / Start / Stop / Restart / Selftest
৬	হেলথ ড্যাশবোর্ড	v_lis_health	কত অনলাইন, আজ কত রেজাল্ট, কত ক্রিটিক্যাল

৫.৬.৩ ফর্মে যে নিয়মগুলো বসাবেন

ফিল্ড	নিয়ম	কেন
analyzer_name	Required, Unique, এডিটে Read Only	নাম বদলালে হিস্টোরি ভেঙে যায়
protocol	LOV: ASTM / HL7 / AUTO	ভুল প্রোটোকল = কোনো রেজাল্ট না
connection_type	LOV: TCP / SERIAL	সিরিয়ালে পোর্ট নম্বর লাগে না
port	1024–65535, TCP হলে Required	ভুল পোর্টে অ্যানালাইজার connect করতে পারবে না
serial_port	SERIAL হলে Required (COM3)	নাহলে পোর্ট খোলা যায় না
enabled	Switch Y/N	বন্ধ = শুধু লিসেনার উঠে যায়, ডেটা থাকে
status	ডিফল্ট DRAFT, শেষে ACTIVE	অর্ধেক লেখা পোর্ট কখনো লাইভ হবে না

> **DRAFT কেন এত জরুরি**
> নতুন অ্যানালাইজার যোগ করার সময় সার্ভিস শুধু `status = 'ACTIVE'` রো পড়ে। তাই আগে DRAFT-এ সব লিখে ফেলুন, মেশিনের Host সেটিং মিলিয়ে নিন, তারপর ACTIVE করুন — তখনই পোর্ট খুলবে। এডিটের সময়ও একই নিয়ম: DRAFT → বদলান → ACTIVE।

৫.৬.৪ বাটনগুলো (lis_commands)

বাটন	কাজ	কখন
RELOAD	সব রো নতুন করে পড়ে, শুধু বদলানোটা রিস্টার্ট	কিছু apply না হলে
START / STOP	একটা অ্যানালাইজার চালু / বন্ধ	সার্ভিসিং বা ভুল করে বন্ধ
RESTART	বন্ধ করে চালু	পোর্ট আটকে গেলে
SELFTEST	ভেতরের নমুনা দিয়ে টেস্ট, ডেটাবেসে লেখে না	নতুন মেশিন যোগ করার পর
PING	সার্ভিস জীবিত কি না	সন্দেহ হলে

`SELFTEST`-এর ফলাফল ওই কমান্ড রো-এর `result_message`-এ ফিরে আসে, যেমন — `astm_cbc_normal: HALF_ACCEPTED, 4 stored, ack=ASTM (E1381 checksum ok)`। ফলে আসল মেশিনে হাত দেওয়ার আগেই বোঝা যায় পার্সিং আর স্টোরেজ ঠিক আছে।

৫.৬.৫ সার্ভিস ইনস্টল (একবার)

```
# Windows (Administrator PowerShell) — NSSM, auto-restart, password in machine env
.\ops\install_service_windows.ps1 -ProjectDir "C:\lis\lis-middleware"
nssm status LISMiddleware

# Linux — systemd unit already in ops/
sudo cp ops/lis-middleware.service /etc/systemd/system/
sudo install -m 600 ops/lis-middleware.env.example /etc/lis-middleware.env   # DB password
sudo systemctl enable --now lis-middleware
journalctl -u lis-middleware -f
```

`config/config.yaml`-এ এখন শুধু ডেটাবেসের ঠিকানা (host / service_name / username), লগ-লেভেল আর ফলব্যাক তালিকা থাকে; পাসওয়ার্ড থাকে সার্ভিসের environment-এ। অ্যানালাইজার, পোর্ট, প্রোটোকল, টেস্ট কোড ম্যাপ — সব ডেটাবেসে।

৫.৬.৬ ভুল হলে কী হয়

পরিস্থিতি	সার্ভিসের প্রতিক্রিয়া	APEX-এ যা দেখবেন
পোর্ট ৮০ লিখেছেন	প্রয়োগ করে না, আগের কনফিগ চালু থাকে	last_error = port outside 1024-65535
দুই মেশিন একই পোর্টে	প্রয়োগ করে না	last_error = port 6661 used by both A and B
পোর্ট অন্য প্রোগ্রাম ধরে আছে	শুধু ওই লিসেনার উঠবে না	state = STOPPED + cannot open port
সব ঠিক	শুধু বদলানো লিসেনার রিস্টার্ট	সবুজ applied tick, state = ONLINE
সার্ভিস বন্ধ	—	heartbeat পুরোনো, preflight ⚠ দেখাবে

নিয়মটা সহজ: **ভুল কনফিগ কখনো চলতি কনফিগকে ভাঙে না।** মেসেজ আসতে থাকে, রেজাল্ট জমতে থাকে, শুধু লাল লেখাটা অপারেটরকে জানায় কী ঠিক করতে হবে।

বিভাগ ৬

সেটআপ — ধাপে ধাপে

৬.১ ধাপ ১ — Oracle স্কিমা তৈরি

```
# Oracle schema (use your own service name)
sqlplus lis_user/password@//192.168.1.100:1521/ORCL @database/schema.sql

# or run schema.sql from SQL Developer / SQLcl
```

৬.২ ধাপ ২ — config.yaml পূরণ

config/config.yaml — সংযোগ ও মেশিন

```
database:
  host: "192.168.1.100"      # DB server IP
  port: 1521
  service_name: "ORCL"       # XEPDB1 / FREEPDB1 / ORCL
  username: "lis_user"
  password: "CHANGE_ME"
analyzers:
  - name: "Sysmex_XN_1000"
    protocol: "ASTM"
    connection_type: "TCP"
    port: 6661
    enabled: true
```

৬.৩ ধাপ ৩ — ইনস্টল ও প্রথম রান

```
cd lis-middleware
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# test the Oracle connection only
python -m database.db_connection

# run without analyzers (results kept in memory)
python main.py --dry-run

# the real thing
python main.py
```

৬.৩.৫ ধাপ ৩.৫ — preflight (ল্যাবে যাওয়ার আগে বাধ্যতামূলক)

এক কমান্ডে সব যাচাই: Python ও প্যাকেজ, config-এর ভুল (ডুপ্লিকেট পোর্ট, ভুল protocol), পোর্ট খোলা যায় কি না, Oracle-এ পৌঁছায় কি না, টেবিল/ভিউ আছে কি না, লগ ফোল্ডার লেখা যায় কি না।

```
python tools/preflight.py            # পুরো চেক (Oracle সহ)
python tools/preflight.py --offline  # DB ছাড়া

# □ থাকলে ল্যাবে যাবেন না — প্রতিটি □-এর নিচে সমাধান লেখা থাকে
# সব ■ হলে: python main.py  → device_sim দিয়ে নকল মেশিন → APEX-এ রো দেখা
```

৬.৪ ধাপ ৪ — মেশিনে সেটিং

analyzer-এর **Host Communication / LIS Connection** পেজে গিয়ে বসান:

ফিল্ড	মান
Host / LIS IP	middleware চলা পিসির স্থির IP
Host Port	6661 (ASTM) / 6662 (HL7) — config-এর সাথে হুবহু
Protocol	ASTM (Sysmex, Roche) / HL7 (Mindray)
Transmission	ASTM হলে E1381 (কিছু মেশিনে Basic/RAW)
Checksum	চালু (আমরা যাচাই করি)
Baud/Data/Parity/Stop	9600 · 7 · EVEN · 1 (RS-232)

ফিল্ড	মান
Sample ID	APEX-এ স্ক্যান করা বারকোডই
Auto-send results	চালু
Host query	প্রথম দিনে বন্ধ

> **Firewall**
> Windows-এ পোর্ট খুলতে ভুলবেন না:`New-NetFirewallRule -DisplayName "LIS 6661" -Direction Inbound -LocalPort 6661 -Protocol TCP -Action Allow`

বিভাগ ৬ক

ল্যাব ডে — ক্রম মেনে

ঘরে সব প্রস্তুত হওয়ার পর ল্যাবে মাত্র এই ধাপগুলো:

1. **পিং ও পোর্ট:** `ping <মেশিনের IP>` এবং `netstat -ano | findstr 6661` — LISTENING আছে কি?
2. **কন্ট্রোল স্যাম্পল:** মেশিন থেকে QC বা ফাঁকা স্যাম্পল পাঠান; লগে `parsed N result(s)` খুঁজুন।
3. **APEX:** `v_sample_results`-এ স্যাম্পল আইডি ও ভ্যালু মিলছে কি?
4. **একটা আসল স্যাম্পল,** তারপর ল্যাবের নিজের রিপোর্টের সাথে মিলিয়ে দেখা।
5. **ক্যাপচার:** `sniff_com` দিয়ে সেশন রেকর্ড করে রাখুন — ভবিষ্যতের টেস্টের ভিত্তি।

> **আটকে গেলে রোলব্যাক**
> মেশিনের Host communication **Off** করে দিন — মেশিন আবার নিজের প্রিন্টারে রিপোর্ট দেবে, ল্যাব আটকে থাকবে না। লগ + `lis_raw_messages` নিয়ে ঘরে ফিরে `device_sim`/`replay_capture` দিয়ে সমস্যা রিপ্রোডিউস করে ঠিক করুন।

বিস্তারিত ধাপ, মেশিনের সেটিং টেবিল আর সমস্যার তালিকা: `docs/LAB_DAY.md`।

বিভাগ ৭

টেস্ট ও মক অ্যানালাইজার

আসল মেশিন ছাড়া টেস্ট করা যায় দুইভাবে — দুটোই প্রজেক্টে আছে।

৭.১ ইউনিট টেস্ট — Oracle ছাড়াই

```
python -m unittest discover -s tests -v
# 50 tests — ASTM, HL7, E1381 framing, validator, dedupe, ACK, hot reload
```

টেস্টগুলো আসল ফরম্যাটের নমুনা দিয়ে চলে (samples/messages.yaml), তাই parser বদলালে সাথে সাথেই বোঝা যায় কিছু ভেঙেছে কি না।

৭.২ মক অ্যানালাইজার — মেশিনের অভিনয়

```
# list scenarios
python tools/mock_analyzer.py --list

# pretend to be a Sysmex: one CBC message
python tools/mock_analyzer.py -s astm_cbc_normal -p 6661

# pretend to be a Mindray: HL7, 5x every 10s (load test)
python tools/mock_analyzer.py -s hl7_biochem -p 6662 --repeat 5 --interval 10

# serial port (use a com0com virtual pair to test)
python tools/mock_analyzer.py -s astm_cbc_normal --serial COM4
```

৭.৩ ব্রাউজার ডেমো

```
python -m demo.app        # http://localhost:8080
```

একই pipeline HTTP দিয়ে চালায় — মেসেজ পাঠানো, ACK দেখা, raw মেসেজ, parsed রেজাল্ট, ডুপ্লিকেট টেস্ট — সব ব্রাউজারে। মেসেজ এডিট করে ইচ্ছাকৃত ভাঙলে error path-ও দেখা যায়।

৭.৪ আসল মেশিনের অনুভূতি — E1381 ওয়্যার লেভেল

ব্রাউজার ডেমো একটা HTTP ফর্ম — আসল অ্যানালাইজার তা নয়। আসল মেশিন ASTM E1381 হ্যান্ডশেক করে: **ENQ → ACK → STX⟨নম্বর⟩ডেটা⟨ETX⟩⟨চেকসাম⟩ → ACK … → EOT**। চেকসাম ভুল হলে মেশিন NAK পায় এবং একই ব্লক আবার পাঠায়। এই লেয়ারটা `core/e1381.py`-তে আছে এবং ৮টি টেস্টে যাচাই করা।

একই কথোপকথন যেটা কেবলে ঘটে — ডেমো ফর্মে দেখা যায় না

```
analyzer                        middleware
  ENQ  (0x05)  -------------->  "আমার পাঠানোর আছে"
               <--------------  ACK  (0x06)
  STX 1 <records> ETX <cs> -->  ফ্রেম নম্বর + ডেটা + চেকসাম
               <--------------  ACK / NAK  (NAK হলে মেশিন রিপিট করে)
  EOT  (0x04)  -------------->  শেষ; এখন পুরো মেসেজ প্রসেস হয়
```

tools/device_sim.py — ওয়্যার-লেভেল সিমুলেটর

```
python main.py --dry-run                                    # টার্মিনাল ১
python tools/device_sim.py -s astm_cbc_normal -p 6661 --trace  # টার্মিনাল ২

# কেবল-এর সমস্যা বানিয়ে দেখা
python tools/device_sim.py -s astm_cbc_normal --fault checksum --fault-rate 0.4
python tools/device_sim.py -s astm_cbc_normal --fault noise    --fault-rate 0.4
python tools/device_sim.py -s astm_cbc_normal --fault drop     --fault-rate 0.25
python tools/device_sim.py -s hl7_critical   -p 6662 --trace
```

৭.৬ Mirth ছাড়া অন্য সিমুলেটর

আসল মেশিনের মতো টেস্ট করার জন্য আরও অপশন আছে — কমার্শিয়াল, ওপেন সোর্স আর ভার্চুয়াল পোর্ট:

টুল	কী ভালো	সীমা
tools/device_sim.py (এই প্রজেক্ট)	E1381 হ্যান্ডশেক, চেকসাম, NAK-রট্রাই, নয়েজ, MLLP, ফল্ট ইনজেকশন	সিনারিও নিজে বানাতে হবে
Lab Analyzer Simulator (ওপেন সোর্স)	ব্রাউজার প্যানেল, HL7+ASTM, Docker, ফিক্সচার	থার্ড-পার্টি, নমুনা ফরম্যাট
Sand Lake Simulator	লো-লেভেল ASTM/HL7 ডিবাগিং, ট্রান্সমিশন ফাইল	কমার্শিয়াল লাইসেন্স
HEDONICS ASTM Simulator	ইনস্ট্রুমেন্ট ↔ LIS দুই দিকেই	কমার্শিয়াল

টুল	কী ভালো	সীমা
HL7 Inspector / 7Edit	HL7 মেসেজ দেখা/এডিট	ASTM বোঝে না
com0com / HHD Virtual Serial Ports	ভার্চুয়াল COM পেয়ার (COM3↔COM4)	ড্রাইভার ইনস্টল

কমার্শিয়াল টুলের দাম/লাইসেন্স বদলাতে পারে — কেনার আগে যাচাই করুন। নিজের কাজের ৯০% `device_sim` দিয়েই হবে।

৭.৫ আসল মেশিনের ডেটা ধরা ও রিপ্লে

সিমুলেটর যতই ভালো হোক, আপনার Sysmex-এর ফিল্ড ম্যাপিং সে জানে না। একবার আসল ক্যাপচার নিলে সেটাই চিরকালের রেফারেন্স:

```
# সিরিয়াল ট্যাপ (RS-232): মেশিন → sniff_com → middleware
python tools/sniff_com.py --port COM4 --baud 9600 --parity EVEN \
    --tap 127.0.0.1:6661 --out logs/capture.jsonl

# TCP ট্যাপ: মেশিনের Host সেটিং-এ পোর্ট 6663 দিন
python tools/sniff_com.py --listen 0.0.0.0:6663 --tap 127.0.0.1:6661

# পরে যতবার চান রিপ্লে করুন (parser বদলালে এটাই রিগ্রেশন টেস্ট)
python tools/replay_capture.py logs/capture.jsonl --list
python tools/replay_capture.py logs/capture.jsonl --port 6661 --timing
```

> **সৎ কথা**
> ডেমো ডেটা দিয়ে শুধু পাইপলাইন প্রমাণ হয়, আপনার মেশিনের ফরম্যাট নয়। তাই আসল মেশিনে যাওয়ার আগে অন্তত একবার `sniff_com` দিয়ে ক্যাপচার নিন এবং সেই সেশন `replay_capture` দিয়ে চালান। বিস্তারিত: `docs/REAL_DEVICE.md`।

সিনারিও	কী পরীক্ষা হয়
astm_cbc_normal	সাধারণ ৪-টেস্ট CBC, patient/sample লিংক
astm_cbc_abnormal	H / L / LL flag, critical শনাক্তকরণ
hl7_biochem	HL7 ORU^R01, OBX পার্সিং
hl7_critical	HH flag → is_critical = Y
astm_merged_unit	ভ্যালু ও ইউনিট একই ফিল্ডে ('5,8 mmol/L')
broken_message	ভাঙা মেসেজ — NAK, কিন্তু raw সংরক্ষিত

বিভাগ ৮

ডিপ্লয়মেন্ট

> **পাসওয়ার্ড ও সিকিউরিটি**
> `config.yaml`-এ প্লেইন পাসওয়ার্ড রাখবেন না — প্রোডাকশনে `LIS_DB_PASSWORD` এনভায়রনমেন্ট ভ্যারিয়েবল ব্যবহার করুন (db_connection.py এটি আগে দেখে)। ফাইল পারমিশন সীমিত রাখুন, এবং ডেটাবেস ইউজারকে শুধু প্রয়োজনীয় টেবিলের ইনসার্ট/সিলেক্ট দিন।

ল্যাবে সাধারণত একটা ছোট পিসি বা সার্ভার ২৪ ঘণ্টা চলে। তাই middleware-ক **সার্ভিস** হিসেবে বসাতে হবে — কেউ লগআউট করলেও চলবে।

৮.১ Windows — সার্ভিস হিসেবে

```
# with NSSM (easiest)
nssm install LISMiddleware "C:\Python311\python.exe" "C:\lis\lis-middleware\main.py"
nssm set LISMiddleware AppDirectory "C:\lis\lis-middleware"
nssm set LISMiddleware AppStdout "C:\lis\logs\stdout.log"
nssm start LISMiddleware

# or with Task Scheduler:
#   Trigger: At startup
#   Action:  python.exe main.py  (Start in: C:\lis\lis-middleware)
```

৮.২ Linux — systemd

```
# /etc/systemd/system/lis-middleware.service
[Unit]
Description=LIS Middleware (ASTM/HL7 to Oracle)
After=network.target

[Service]
User=lis
WorkingDirectory=/opt/lis-middleware
ExecStart=/opt/lis-middleware/.venv/bin/python main.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now lis-middleware
journalctl -u lis-middleware -f
```

> **প্রোডাকশনের আগে**
> ১) `--dry-run` দিয়ে অন্তত একদিন চালান। ২) লগ ফাইল ঘুরছে কি না দেখুন (RotatingFileHandler, ৫ MB × ৫)। ৩) ডেটাবেস ব্যাকআপ রাখুন। ৪) `tools/replay.py --status ERROR` দিয়ে ব্যর্থ মেসেজ দেখুন।

বিভাগ ৯

সমস্যা ও সমাধান

লক্ষণ	কারণ	সমাধান
connection refused	middleware চলছে না বা পোর্ট আলাদা	লগে 'listening on' লাইন দেখুন, config মেলান
ORA-12541 (no listener)	host/port/service_name ভুল	python -m database.db_connection দিয়ে টেস্ট
ORA-01017 (invalid login)	ইউজারনেম/পাসওয়ার্ড ভুল	config.yaml ঠিক করুন
no results parsed	protocol ভুল লেবেল	protocol: AUTO দিন, লগে detected_protocol দেখুন
সিরিয়ালে আজেবাজে অক্ষর	baud/parity মেলেনি	9600 7E1, তারপর 9600 8N1 চেষ্টা করুন
মেশিন কিছু পাঠাচ্ছেই না	ACK-এর অপেক্ষায় আছে	HL7 হলে app.ack_mode: MLLP দিন
একই রেজাল্ট দুইবার	মেশিন পুনরায় পাঠাচ্ছে (স্বাভাবিক)	ডুপ্লিকেট আটকায়; lis_raw_messages-এ DUPLICATE
QC নমুনা রিপোর্টে ঢুকছে	quality-control স্যাম্পল আসল রোগীর মতোই আসে	validation.ignore_sample_prefixes-এ QC/CTRL/CAL দিন → status IGNORED
পোর্ট ইতিমধ্যে ব্যবহৃত	অন্য প্রোগ্রাম	netstat -ano | findstr 6661 → PID বন্ধ করুন
লগে checksum mismatch	কেবলের নয়েজ বা parity ভুল	লিসেনার NAK দেয়, মেশিন আবার পাঠায় — 7E1 মিলিয়ে নিন

লক্ষণ	কারণ	সমাধান
মেশিন মাঝপথে থেমে যায়	ব্লকের ACK-এর অপেক্ষায়	লগে bad block → NAK দেখুন; এটাই রিট্রাই পথ

৯.১ লগ পড়ার নিয়ম

সুস্থ সিস্টেমের লগ — প্রথম লাইন থেকেই বোঝা যায় কোথায় আটকেছে

```
2026-11-04 10:30:01 | INFO  | LIS.DB   | Oracle DB connected: 192.168.1.100:1521/ORCL
2026-11-04 10:30:01 | INFO  | LIS.TCP  | Sysmex_XN_1000 | TCP | listening on 0.0.0.0:6661
2026-11-04 10:30:12 | INFO  | LIS.TCP  | Sysmex_XN_1000: received 412 bytes from 192.168.1.50
2026-11-04 10:30:12 | INFO  | LIS.ASTM | ASTM: parsed 4 result(s) from Sysmex_XN_1000
2026-11-04 10:30:12 | INFO  | LIS.DBW  | inserted SAMPLE-0001 / WBC = 7.5 10*3/uL
2026-11-04 10:30:12 | WARN | LIS.DBW  | duplicate skipped: SAMPLE-0001 / RBC
```

বিভাগ ১০

শেষ চেকলিস্ট

১০.১ ডেটাবেস

- schema.sql চালানো হয়েছে, TEAM লিস্টে `lis_user` আছে
- `python -m database.db_connection` → OK
- ইনডেক্স আছে (sample_id, received_at, is_critical)
- ব্যাকআপ শিডিউল করা আছে

১০.২ Middleware

- `python -m unittest discover -s tests` → সব পাস
- `--dry-run` দিয়ে একদিন চলেছে
- সার্ভিস হিসেবে ইনস্টল করা (Windows NSSM / Linux systemd)
- লগ রোটেট হচ্ছে, ডিস্ক ভরছে না
- সার্ভার রিস্টার্ট হলে নিজে থেকে চালু হয়

১০.৩ মেশিন

- প্রতিটি analyzer-এর IP, পোর্ট, প্রোটোকল কাগজে লিখে রাখা
- ACK mode (ASTM / MLLP) মিলিয়ে দেখা
- একটা আসল স্যাম্পল দিয়ে end-to-end টেস্ট
- বারকোড স্ক্যান → মেশিন → APEX — পুরো পথে ভ্যালু মিলেছে
- আসল মেশিনের এক ক্যাপচার নেওয়া হয়েছে এবং সেটি রিপ্লে করে দেখা গেছে
- `python tools/preflight.py` → কোনো □ নেই (ল্যাবে যাওয়ার আগের শেষ গেট)
- সার্ভারে স্থির IP + ফায়ারওয়ালে পোর্ট খোলা
- মেশিনের Host সেটিং-এর নাম আর মান খাতায় লেখা (ম্যানুয়ালের Host Interface অধ্যায়)

১০.৪ APEX

- Interactive Report + Form পেজ তৈরি
- critical রেজাল্টের জন্য আলাদা রিপোর্ট/অ্যালার্ট
- অ্যাবনর্মাল ফ্ল্যাগে রঙ হাইলাইট
- ইউজার পারমিশন (ল্যাব টেকনিশিয়ান vs ডক্টর)

শেষ কথা

মিডলওয়্যার শুধু অনুবাদক — মেশিন যা মাপে, তাই ডেটাবেসে পৌঁছায়, কিছু বাড়ে না কমে না। তাই প্রতিটি নতুন মেশিন যোগ করার সময় একটা ম্যানুয়াল রেজাল্টের সাথে মিলিয়ে নিন।